

BAHAN AJAR PERTEMUAN 1: FONDASI ALGORITMA, MASALAH, DAN EKSPRESI EVALUASI

1. AT A GLANCE: TIK PERTEMUAN 1

- **Capaian Pembelajaran:** Mahasiswa mampu mendefinisikan konsep masalah dan algoritma secara formal, memahami tahapan siklus pengembangan program, serta mengoperasikan operasi evaluasi dasar pada lingkungan REPL.
 - **Durasi:** 3 Jam Kuliah (150 Menit).
 - **Metode:** Kuliah Teoretis, Bedah Kasus Logika, dan Praktikum REPL Terbimbing.
-

2. MATERI TEORITIS UTAMA

A. Pengertian Masalah dan Penyelesaiannya (Problem Solving)

Dalam ranah ilmu komputer, **masalah** (*problem*) didefinisikan secara formal sebagai suatu situasi atau kondisi di mana kita memiliki titik awal (*data awal/input*), seperangkat aturan atau batasan yang mengikat, dan adanya kebutuhan untuk mencapai tujuan akhir yang diinginkan (*output/solusi*). Dengan kata lain, masalah adalah kesenjangan antara kondisi saat ini dengan kondisi ideal yang ingin dicapai.

Aktivitas **pemecahan masalah** (*problem solving*) merupakan sebuah proses sekuensial untuk menganalisis informasi terkait situasi tersebut guna menghasilkan opsi respons yang tepat demi mengatasi hambatan menuju target yang dituju. Langkah awal yang paling krusial adalah mendefinisikan masalah itu sendiri, yaitu dengan mengidentifikasi secara presisi apa yang kita ketahui (*input*) dan apa yang ingin kita hasilkan (*output*).

B. Definisi dan Karakteristik Algorithmic

Algoritma adalah prosedur atau urutan langkah-langkah abstrak yang disusun secara logis dan sistematis untuk menyelesaikan suatu masalah tanpa terikat pada bahasa pemrograman tertentu. Istilah algoritma sendiri diserap dari nama seorang matematikawan Persia, Muhammad ibn Mūsā al-Khwārizmī (sekitar 800 CE).

Sebuah prosedur dapat dikategorikan sebagai algoritma yang baik jika memenuhi karakteristik berikut:

- **Definiteness (Kepastian):** Setiap langkah harus didefinisikan secara jelas, spesifik, dan tidak menimbulkan makna ganda (non-ambigu).
- **Finiteness (Keterbatasan):** Algoritma harus memiliki titik henti yang jelas; ia harus

berhenti setelah mengeksekusi sejumlah langkah terbatas.

- **Input:** Memiliki nol atau lebih data masukan yang diberikan di awal sebelum diproses.
- **Output:** Menghasilkan minimal satu atau lebih data keluaran sebagai solusi.
- **Effectiveness (Efektivitas):** Setiap langkah harus cukup sederhana sehingga dapat dikerjakan dalam waktu yang masuk akal.

C. Siklus Pengembangan Program (Program Development Cycle)

Mengubah algoritma abstrak menjadi perangkat lunak yang berjalan di komputer memerlukan proses terstruktur yang disebut Siklus Pengembangan Program. Tahapan utamanya meliputi:

1. **Analisis Masalah (Problem Analysis):** Memahami kebutuhan pengguna, menentukan spesifikasi input, dan menetapkan output yang diharapkan.
2. **Desain Algoritma (Algorithm Design):** Merancang alur logika penyelesaian menggunakan alat bantu seperti diagram alur (*flowchart*) atau bahasa semu (*pseudocode*).
3. **Pengodean (Coding/Implementation):** Menerjemahkan rancangan algoritma ke dalam sintaksis bahasa pemrograman tertentu (misalnya Python).
4. **Pengujian dan Validasi (Testing & Validation):** Menjalankan program dengan berbagai variasi kasus uji (*test cases*) untuk mendeteksi kesalahan sintaksis, logika, maupun waktu runtutan (*runtime*).
5. **Dokumentasi dan Pemeliharaan (Documentation & Maintenance):** Menulis catatan penjelasan kode program dan memperbarui sistem secara berkala.

3. BEDAH LOGIKA: STUDI KASUS DAN EKSPRESI EVALUASI REPL

Untuk memahami bagaimana komputer mengevaluasi instruksi, kita akan menggunakan lingkungan **REPL (Read-Evaluate-Print Loop)**. REPL bekerja dalam siklus abadi: membaca baris perintah yang diketik pengguna, mengevaluasinya secara internal, mencetak hasilnya ke layar, lalu kembali menunggu perintah baru.

Dalam pertemuan pertama ini, mahasiswa wajib membedakan tiga jenis mekanisme evaluasi dasar komputer:

1. Evaluasi Konstanta Literal (Self-Evaluation)

Ketika sebuah konstanta mentah dimasukkan ke dalam sistem, komputer akan langsung mengembalikan nilai tersebut tanpa modifikasi karena nilainya sudah bersifat absolut.

- **Simulasi Perintah di Prompt REPL:**

```
Python
>>> 15
15
```

```
>>> 3.14
3.14
```

- **Analisis Logika:** Angka 15 dievaluasi sebagai tipe bilangan bulat (int), sedangkan 3.14 dievaluasi sebagai bilangan pecahan (float). Keduanya adalah komponen dasar konstan.

2. Evaluasi Ekspresi Majemuk (Expression Evaluation)

Komputer mengevaluasi sekelompok elemen sintaksis yang digabungkan dengan operator matematika untuk menghasilkan satu nilai tunggal. Urutan pengerjaannya tunduk pada aturan prioritas matematika universal (**BODMAS**: *Brackets, Orders, Division/Multiplication, Addition/Subtraction*).

- **Simulasi Perintah di Prompt REPL:**

Python

```
>>> (5 * 6) + 4
34
>>> 5 * (6 + 4)
50
```

- **Analisis Logika:**
 - Pada baris pertama, perkalian $5 * 6$ dilakukan terlebih dahulu menghasilkan 30, kemudian ditambahkan 4 sehingga menghasilkan nilai akhir 34.
 - Pada baris kedua, penggunaan tanda kurung memaksa operasi penjumlahan $6 + 4$ berjalan di awal menghasilkan 10, yang kemudian dikalikan dengan 5 menghasilkan nilai akhir 50.

3. Evaluasi Simbol Variabel (Symbol Evaluation)

Komputer mencari nilai yang tersimpan di dalam memori yang telah diasosiasikan dengan sebuah nama pengenalan tertentu (*identifier*). Jika nama tersebut belum pernah didaftarkan, komputer akan memicu galat teks kesalahan (*Error Report*).

- **Simulasi Perintah di Prompt REPL:**

Python

```
>>> x = 10
>>> x
10
>>> y
NameError: name 'y' is not defined
```

- **Analisis Logika:**
 - Sintaksis $x = 10$ memerintahkan memori untuk menyimpan literal 10 ke dalam pengenalan bernama x. Saat kita mengetik x, komputer melakukan pencarian simbol dan mengembalikan nilai 10.
 - Saat kita mengetik y, evaluasi gagal karena simbol y belum dialokasikan di dalam tabel memori sistem, sehingga memicu NameError.
-

4. LEMBAR AKTIVITAS PRAKTIKUM MAHASISWA (LAB SHEET)

Instruksi: Buka aplikasi terminal atau konsol Python di komputer Anda, jalankan mode interaktif, dan selesaikan tantangan evaluasi logika berikut. Catat hasil keluarannya.

1. **Tantangan Prioritas Operator:** Ketik ekspresi berikut secara persis dan jelaskan mengapa hasilnya berbeda:

Python

```
>>> 10 + 2 * 5
```

```
>>> (10 + 2) * 5
```

2. **Eksperimen Pembagian:** Amati perilaku operator pembagian tunggal / dibandingkan dengan pembagian lantai //:

Python

```
>>> 11 / 4
```

```
>>> 11 // 4
```

```
>>> 11 % 4
```

Tugas: Jelaskan fungsi dari masing-masing operator /, //, dan % berdasarkan hasil keluaran yang muncul di layar!

Aktivitas Pertemuan 1 ini berfokus sepenuhnya pada penguasaan cara berpikir logis komputer sebelum kita masuk ke pembuatan diagram alur terstruktur di minggu depan.

Sesi materi berikutnya siap dilaksanakan. Apakah Anda ingin saya men-generate draf **Soal Kuis Teoretis Pilihan Ganda** beserta kunci jawabannya khusus untuk materi Pertemuan 1 ini?

Sources:

- [Principle Algorith dan Programming.pdf](#)
- [Basics of Programming and Algorithms, Prin - Roberto Mantaci.pdf](#)
- [Konsep bahan Ajar Algoritma dan Pemrograman](#)

Sesi & Durasi	Topik Pembahasan	Estimasi Slide
Pemanasan (15 Menit)	Opening & Ice Breaking: • Perkenalan dosen & kontrak kuliah singkat. • <i>Hook</i>: Menanyakan aplikasi favorit mahasiswa dan menjelaskan bahwa semuanya berawal dari algoritma.	3 Slide - Cover materi - Kontrak/Aturan main - Pertanyaan pemantik
Materi 1 (20 Menit)	Apa Itu Komputer & Komputasi? • Analogi komputer sebagai "Asisten Polos". • Konsep dasar: <i>Input</i> $\rightarrow$ <i>Process</i> $\rightarrow$ <i>Output</i>.	3 Slide - Ilustrasi asisten robot - Diagram Input-Proses-Output

		- Contoh sehari-hari
Materi 2 (25 Menit)	Algoritma: Resep Rahasia Teknologi • Penjelasan konsep dasar algoritma dan pemrograman. • Analogi membuat mie instan (langkah demi langkah). • Bahaya <i>Logic Error</i> (bumbu dimasak, mie dibuang).	4 Slide - Definisi <i>fun</i> algoritma - Analogi resep makanan - Contoh <i>logic error</i> lucu - Mengapa urutan itu penting

Materi 3 (25 Menit)	Logika Pemrograman & <i>Problem Solving</i> • <i>Computational Thinking</i> (memecah masalah besar ke kecil). • Analogi membuat kopi (Orang awam vs. <i>Programmer</i>). • 4 Langkah sakti: <i>Analyze, Design, Implement, Test</i>.	4 Slide - Apa itu <i>Computational Thinking</i> - Analisis bikin kopi - 4 Tahapan <i>Problem Solving</i> - Ilustrasi siklus <i>coding</i>
Interaksi (20 Menit)	Mini Quiz & Diskusi (Asah Otak!) • Permainan teka-teki menyeberangkan Serigala, Kambing, dan Kubis. • Mengajak mahasiswa berdiskusi merumuskan	2 Slide - Visual teka-teki serigala - Slide kosong untuk menulis

	"Algoritma Menyeberang" secara bersama-sama.	jawaban mahasiswa bareng-bareng
Penutup (15 Menit)	Kesimpulan & Info Minggu Depan • <i>Review</i> singkat 3 poin utama hari ini. • Kisi-kisi minggu depan: "Kita akan belajar bahasa alien (<i>Pseudocode</i>) dan menggambar (<i>Flowchart!</i>)"	2 Slide - <i>Summary</i> poin penting - <i>Spoiler</i> minggu depan & Penutup